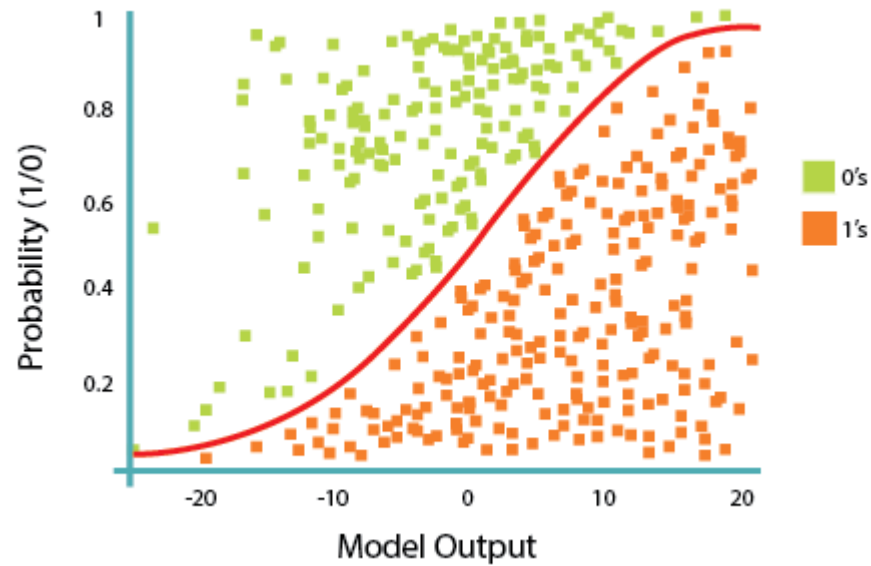


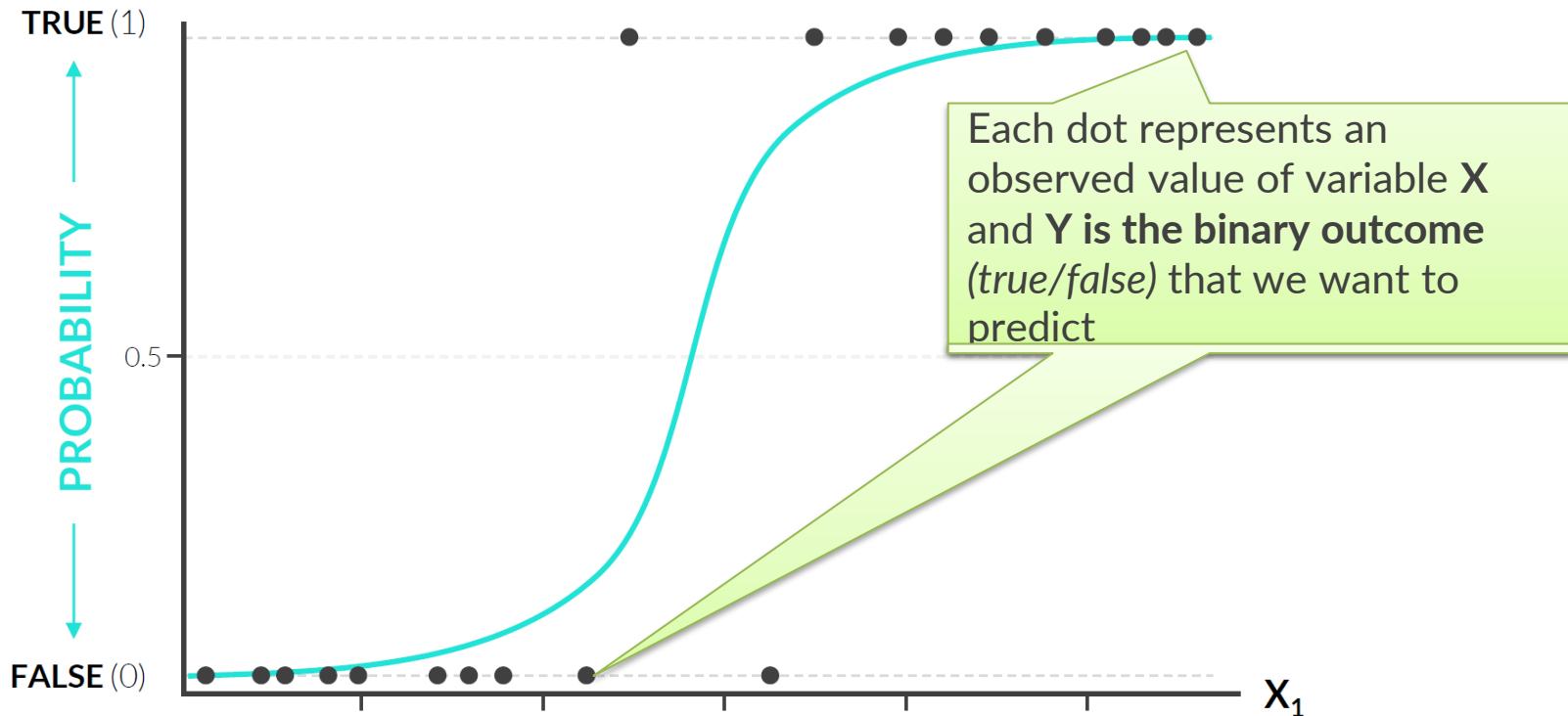
Logistic Regression



Logistic Regression

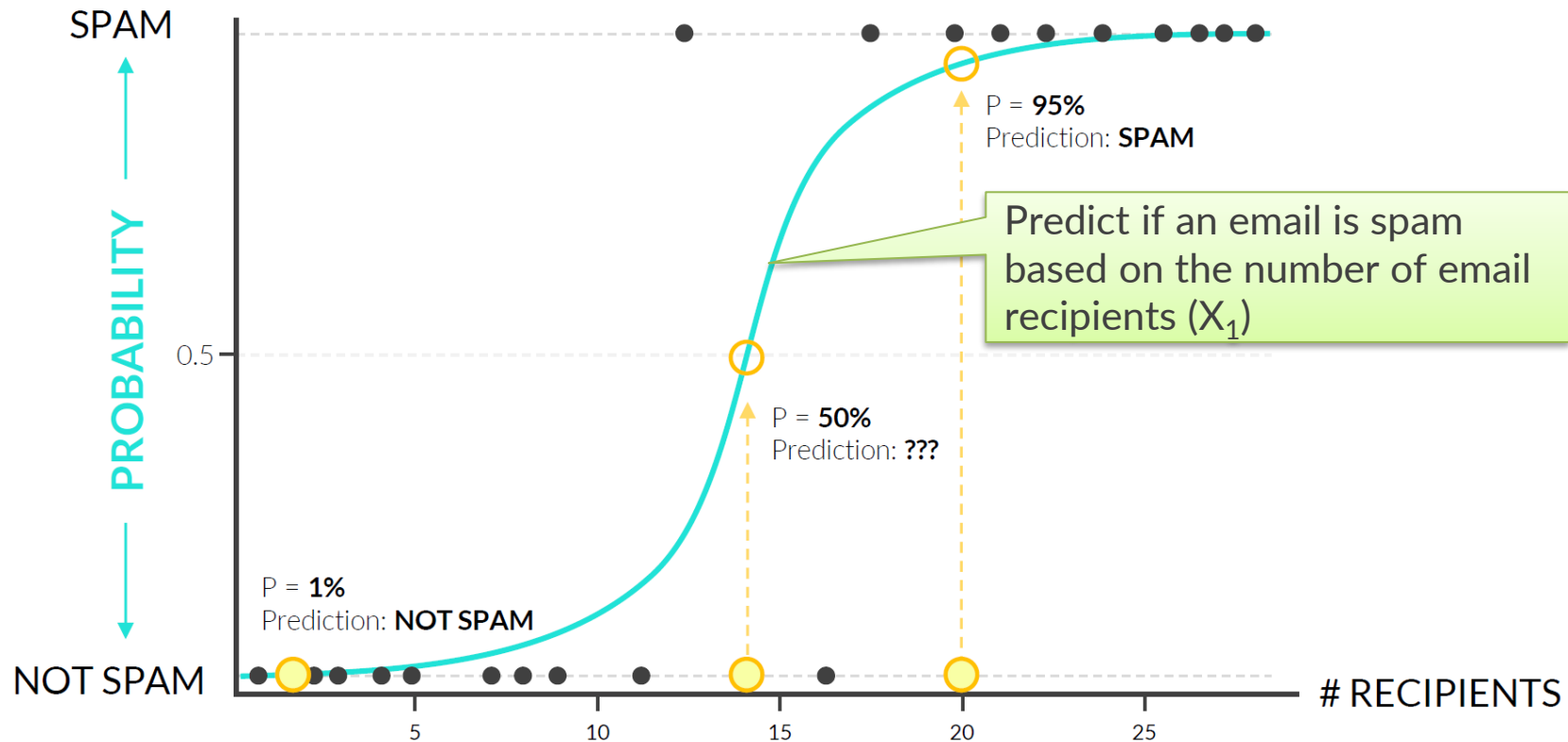
- Logistic Regression is a supervised **classification** algorithm used to model the **probability of a binary outcome** (e.g., True/False, Yes/No, 1/0)
- Despite its name, **logistic regression is not used to predict continuous numeric values**
 - It predicts **probabilities**, which are then converted into class labels
- The model computes a **linear combination of input features** and passes it through a **sigmoid (logistic) function**, producing an output between **0 and 1**
 - This results in an **S-shaped (sigmoid) curve**, representing the probability of the positive class
 - A **decision boundary** (often at 0.5) is used to convert probabilities into final class predictions

Logistic Regression



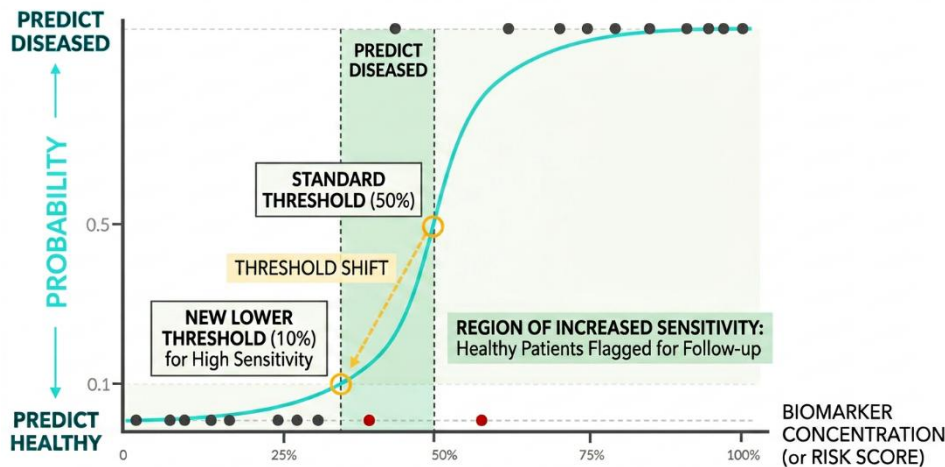
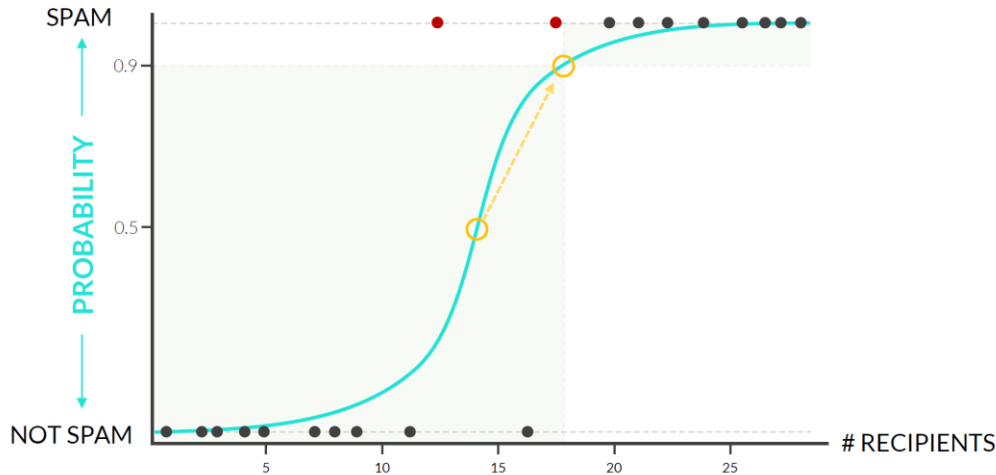
Logistic regression learns an **S-shaped curve** bounded between 0 and 1, representing the probability that **$Y = \text{TRUE}$** for a given input feature **X_1**

Logistic Regression - Example



- Using this model, we can classify unobserved values of X_1 (number of recipients) to predict the probability that Y is true or false (i.e., the probability that an email is spam)
- In practice a threshold of 0.5 is a common a decision point for logistic models ($P > 0.5$ means Y is predicted to be True)

Is 50% Always the Right Decision Threshold?



- Threshold selection is a **business decision**, not a modeling decision

Example 1: Spam Detection

Cost of false positive is high

(Legitimate email marked as spam)

- ✓ Use a **higher threshold** (e.g., 90%)
 - Be very confident before labeling an email as spam
 - Missing a few spam emails is acceptable

Example 2: Medical Screening (e.g., Cancer Detection)

Cost of false negatives is very high

(Patient is predicted healthy but actually has the disease)

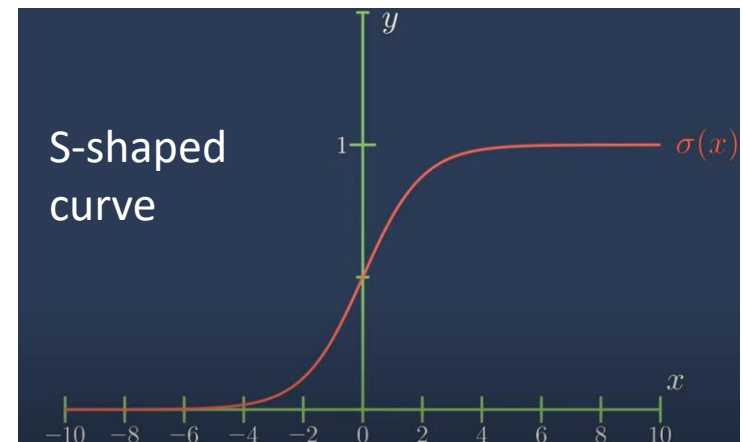
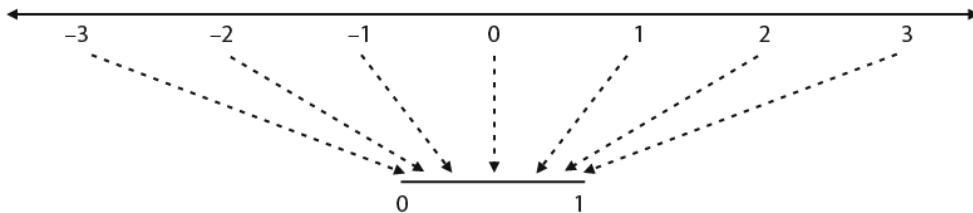
- ✓ Use a **lower decision threshold** (e.g., 10–20%)
- Catch as many true cases as possible
- Some healthy patients may be flagged for follow-up tests
- Extra tests are acceptable compared to missing a real case

Sigmoid (Logistic) Function

- The sigmoid function, also called the **logistic function**, converts model outputs into probabilities (a value between 0 and 1)
- It is denoted by the Greek letter **σ (sigma)** and defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **e** is a special math number (≈ 2.718) that helps turn values into smooth curves, such as probabilities between 0 and 1



Logistic Regression - Model Representation

- Logistic regression applies a sigmoid function to a
- Given training data (x, y) , logistic regression learns a function $f(x)$ that predicts the probability of the positive class.
- The model learns:
 - **Weights** $\mathbf{w}$ (feature importance)
 - **Bias** b

Linear Model

$$z = \sum_{i=1}^n w_i x_i + b \quad \xrightarrow{\text{Vector Notation}} \quad z = \mathbf{w} \cdot \mathbf{x} + b$$

$\mathbf{w}$ and $\mathbf{x}$ are vectors

Probability Prediction

- The linear output z is converted into a probability using the **sigmoid function**:

$$\hat{y} = f(x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- The sigmoid function ensures predictions are always between **0 and 1**

Learning a Logistic Regression Model

- The goal is to learn **weights** $\mathbf{w} = [w_1, \dots, w_m]$ and **bias** b by minimizing the **Binary Cross-Entropy cost function**

$$J = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

- $y^{(i)}$: true label (0 or 1) for example i
- $\hat{y}^{(i)}$: predicted probability that example i belongs to class 1

Intuition

- Measures how close predicted probabilities are to the true labels
- Penalizes heavily confident wrong predictions
- Encourages:
 - $\hat{y} \rightarrow 1$ when $y = 1$
 - $\hat{y} \rightarrow 0$ when $y = 0$

Gradient Descent for Logistic Regression

Objective:

Learn the parameters $\mathbf{w}$ and b that minimize the cost function $J(\mathbf{w}, b)$.

$$\min_{\mathbf{w}, b} J(\mathbf{w}, b)$$

Algorithm Overview

1. Initialize weights $\mathbf{w}$ and bias b (e.g., zeros)

2. Predict probabilities using the model:

$$\hat{y}^{(i)} = \sigma(\mathbf{w} \cdot \mathbf{x}^{(i)} + b)$$

3. Compute gradients of the cost function

4. Update parameters using gradient descent

5. Repeat until the cost converges

Gradients (Binary Cross-Entropy Loss)

For m training examples:

Gradient w.r.t. weights

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

Gradient w.r.t. bias

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$$

Parameter Updates

Using learning rate α :

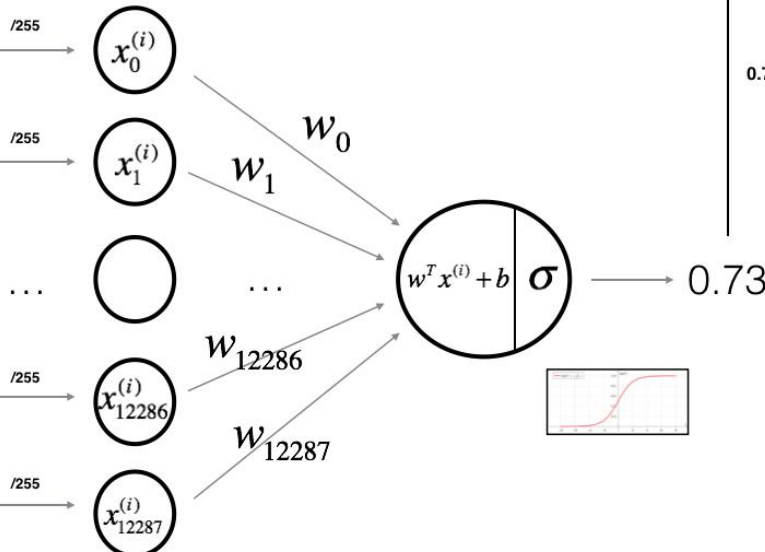
$$\begin{aligned} \mathbf{w} &= \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}} \\ b &= b - \alpha \frac{\partial J}{\partial b} \end{aligned}$$





image2vector

$\begin{pmatrix} 255 \\ 231 \\ \dots \\ 94 \\ 142 \end{pmatrix}$



$0.73 > 0.5$

"it's a cat"

Logistic Regression as a Neural Network

- **Input layer:** An input image is converted into a **feature vector** (pixel values), typically normalized to $[0, 1]$.
- **Single neuron (no hidden layers):** Each input feature x_i is multiplied by a weight w_i , summed, and combined with a bias b :

$$z = \mathbf{w}^T \mathbf{x} + b$$

- **Activation (sigmoid):** The sigmoid function converts z into a probability:

$$\hat{y} = \sigma(z) \in [0, 1]$$